

JOBSHEET XI

<https://github.com/onlyyourza/PraktikumASD>

QUEUE

1. Learning Objectives

After completing this practical session, students will be able to:

1. Understand the structure and concept of the Queue data structure
2. Create and declare a Queue data structure
3. Implement Queue algorithms using arrays

2. Labs 1

2.1 Experiment 1 : Queue Basic Operations

Allocated Time : 90 minutes

In this experiment, we will implement the basic operations of the Queue algorithm. Take a look at the following Queue class diagram:

Queue
data: int[] front: int rear: int size: int max: int
Queue(n: int) isFull(): boolean isEmpty(): boolean enqueue(dt: int): void dequeue(): int peek: void print(): void clear(): void

The following steps will implement Java program from the above design.

2.1.1. Experiment Steps

1. Create a new folder named **P1Jobsheet11** or another preferred name, inside the Praktikum-ASD repository, then create a new class named **Queue**.
2. Add the attributes of the **Queue** as shown in the class diagram, then add the constructor as described below:

```
int[] data;
int front, rear, size, max;

public Queue(int n){
    max = n;
}
```

```
data = new int[max];
size = 0;
front = rear = -1;
}
```

3. Create a boolean method named **isEmpty** to check whether the queue is empty.

```
boolean isEmpty(){
    if(size == 0){
        return true;
    }else{
        return false;
    }
}
```

4. Create a boolean method named **isFull** to check whether the queue is full.

```
boolean isFull(){
    if(size == max){
        return true;
    }else{
        return false;
    }
}
```

5. Create a void method named **peek** to display the element at the **front** of the queue.

```
void peek(){
    if(!isEmpty()){
        System.out.println("Front data: "+data[front]);
    }else{
        System.out.println("Queue is empty!!!");
    }
}
```

6. Create a void method named **print** to display all elements in the queue, from the **front** to the **rear** position.

```
void print(){
    if(!isEmpty()){
        int i = front;
        while(i != rear){
            System.out.print(data[i]+" ");
            i = (i+1)%max;
        }
        System.out.println(data[i]+" ");
        System.out.println("Number of element: "+size);
    }else{

```

```

        System.out.println("Queue is empty!!!");
    }
}

```

7. Create a void method named **clear** to remove all elements from the queue.

```

void clear(){
    if(!isEmpty()){
        front = rear = -1;
        size = 0;
        System.out.println("All data has been successfully
removed!");
    }else{
        System.out.println("Queue is already empty!!!");
    }
}

```

8. Create a void method named **enqueue** to add an element to the queue, using a parameter **dt** of type **int**.

```

void enqueue(int dt){
    if(!isFull()){
        if(isEmpty()){
            front=rear=0;
        }else{
            if(rear==max-1){
                rear=0;
            }else{
                rear++;
            }
        }
        data[rear]=dt;
        size++;
        System.out.printf("%d is successfully added at index %d\n",
dt, rear);
    }else{
        System.out.println("Queue is full!!!");
    }
}

```

9. Create an **int** method named **dequeue** to remove and return the data from the **front** position of the queue.

```

int dequeue(){
    int dt = 0;
    if(!isEmpty()){
        dt = data[front];
        size--;
    }
}

```

```

        if(isEmpty()){
            front=rear=-1;
        }else{
            if(front==max-1){
                front=0;
            }else{
                front++;
            }
        }
    }else{
        System.out.println("Queue is empty!!!");
    }
    return dt;
}

```

10. Next, create a new class named **QueueMain**. Create a **void** method named **menu** to display and handle the program's menu options during execution.

```

static void menu(){
    System.out.println("Available menu:");
    System.out.println("1. Enqueue");
    System.out.println("2. Dequeue");
    System.out.println("3. Print");
    System.out.println("4. Peek");
    System.out.println("5. Clear");
    System.out.println("=====");
}

```

11. Create the **main** method, then declare a **Scanner** object named **sc**.
12. Create a variable named **n** to store the user input representing the maximum number of elements that can be stored in the queue.

```

Scanner sc = new Scanner(System.in);
System.out.print("Input maximum number of data: ");
int n = sc.nextInt();

```

13. Instantiate a **Queue** object named **Q**, passing **n** as the queue's element capacity.

```

Queue Q = new Queue(n);

```

14. Declare an integer variable named **choice** to store the user's menu selection.
15. Use a **do-while** loop to keep the program running based on user input. Inside the loop, use a **switch-case** statement to execute queue operations according to the user's menu selection.

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    System.out.print("Input maximum number of data: ");
    int n = sc.nextInt();
    Queue Q = new Queue(n);
}

```

```
int choice = -1;
do {
    menu();
    choice = sc.nextInt();
    switch (choice) {
        case 1:
            System.out.print("Input new data: ");
            int newData = sc.nextInt();
            Q.enqueue(newData);
            break;
        case 2:
            int outData = Q.dequeue();
            if (outData != 0) {
                System.out.println("Removed data: " + outData);
                break;
            }
        case 3:
            Q.print();
            break;
        case 4:
            Q.peek();
            break;
        case 5:
            Q.clear();
            break;
    }
} while (choice==1 || choice==2 || choice==3 || choice==4 ||
choice==5);
}
```

16. Compile and run the **QueueMain** class, then observe the results.



```
Jobsheet11 > Queue14.java > {} Jobsheet11
1  package Jobsheet11;
2
3  public class Queue14 {
4      int[] data;
5      int front, rear, size, max;
6
7      // Constructor: menyiapkan array & kondisi awal queue kosong
8      public Queue14(int n) {
9          max = n;
10         data = new int[max];
11         size = 0;
12         front = rear = -1;
13     }
14
15     // Cek apakah queue kosong
16     boolean isEmpty() {
17         if (size == 0) {
18             return true;
19         } else {
20             return false;
21         }
22     }
23
24     // Cek apakah queue penuh
25     boolean isFull() {
26         if (size == max) {
27             return true;
28         } else {
29             return false;
30         }
31     }
32
33     // Menampilkan elemen paling depan
34     void peek() {
35         if (!isEmpty()) {
36             System.out.println("Front data: " + data[front]);
37         } else {
```

```
3 public class Queue14 {
31 }
32
33 // Menampilkan elemen paling depan
34 void peek() {
35     if (!isEmpty()) {
36         System.out.println("Front data: " + data[front]);
37     } else {
38         System.out.println(x: "Queue is empty!!!");
39     }
40 }
41
42 // Menampilkan seluruh elemen dari front ke rear
43 void print() {
44     if (!isEmpty()) {
45         int i = front;
46         while (i != rear) {
47             System.out.print(data[i] + " ");
48             i = (i + 1) % max;
49         }
50         System.out.println(data[i] + " ");
51         System.out.println("Number of element: " + size);
52     } else {
53         System.out.println(x: "Queue is empty!!!");
54     }
55 }
56
57 // Mengosongkan queue
58 void clear() {
59     if (!isEmpty()) {
60         front = rear = -1;
61         size = 0;
62         System.out.println(x: "All data has been successfully removed!");
63     } else {
64         System.out.println(x: "Queue is already empty!!!");
65     }
66 }
```

```
Jobsheet11 > Queue14.java > {} Jobsheet11
3  public class Queue14 {
68      // Menambah elemen ke belakang queue (circular)
69      void enqueue(int dt) {
70          if (!isFull()) {
71              if (isEmpty()) {
72                  front = rear = 0;
73              } else {
74                  if (rear == max - 1) {
75                      rear = 0;
76                  } else {
77                      rear++;
78                  }
79              }
80              data[rear] = dt;
81              size++;
82              System.out.printf(format: "%d is successfully added at index %d\n", dt, rear);
83          } else {
84              System.out.println(x: "Queue is full!!!");
85          }
86      }
87
88      // Mengambil & menghapus elemen dari depan queue (circular)
89      int dequeue() {
90          int dt = 0;
91          if (!isEmpty()) {
92              dt = data[front];
93              size--;
94              if (isEmpty()) {
95                  front = rear = -1;
96              } else {
97                  if (front == max - 1) {
98                      front = 0;
99                  } else {
100                      front++;
101                  }
102              }
103          } else {
```




```

Jobsheet11 / QueueMain14.java / QueueMain14 / main(String[])
1  package Jobsheet11;
2
3  import java.util.Scanner;
4
5  public class QueueMain14 {
6      static void menu() {
7          System.out.println(x: "Available menu:");
8          System.out.println(x: "1. Enqueue");
9          System.out.println(x: "2. Dequeue");
10         System.out.println(x: "3. Print");
11         System.out.println(x: "4. Peek");
12         System.out.println(x: "5. Clear");
13         System.out.println(x: "=====");
14     }
15
16     Run | Debug
17     public static void main(String[] args) {
18         Scanner sc = new Scanner(System.in);
19         System.out.print(s: "Input maximum number of data: ");
20         int n = sc.nextInt();
21         Queue14 Q = new Queue14(n);
22         int choice = -1;
23         do {
24             menu();
25             choice = sc.nextInt();
26             switch (choice) {
27                 case 1:
28                     System.out.print(s: "Input new data: ");
29                     int newData = sc.nextInt();
30                     Q.enqueue(newData);
31                     break;
32                 case 2:
33                     int outData = Q.dequeue();
34                     if (outData != 0) {
35                         System.out.println("Removed data: " + outData);
36                     }
37                     break;
38                 case 3:
39                     Q.print();
40                     break;
41                 case 4:
42                     Q.peek();
43                     break;
44                 case 5:
45                     Q.clear();
46                     break;
47             }
48             // sengaja TIDAK ada break di sini (sesuai jobsheet)
49         } while (choice == 1 || choice == 2 || choice == 3 || choice == 4 || choice == 5);
50         sc.close();
51     }
52

```



2.1.2. Output Verification

Verify the output with the following:

```
Input maximum number of data: 4
Available menu:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
=====
1
Input new data: 15
15 is successfully added at index 0
Available menu:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
```

```
=====
1
Input new data: 31
31 is successfully added at index 1
Available menu:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
```

```
=====
3
15 31
Number of element: 2
Available menu:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
```

```
=====
4
Front data: 15
Available menu:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
```

```
=====
2
Removed data: 15
Available menu:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
```

```
=====
3
31
Number of element: 1
Available menu:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
=====
```



```
PS C:\Users\pasap\OneDrive\Documents\Kuliah\Tugas\Semester 2\Praktik Algoritma Struktur Data\Jobsheet1-semester2> & 'C:\Program
\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\pasap\AppData\Roaming\Code\User\workspaceSt
e0ae0740bf9eef90a1485cc4c4061a4f\redhat.java\jdt_ws\Jobsheet1-semester2_bc378dc3\bin' 'Jobsheet11.QueueMain14'
Input maximum number of data: 20
Available menu:
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
=====

```

2.1.3. Question

1. In the constructor, why are the initial values of the **front** and **rear** attributes set to **-1**, while the size attribute is set to **0**?
2. In the **Enqueue** method, explain the meaning and purpose of the following code snippet!

```
if (rear == max - 1) {
    rear = 0;
}
```

3. In the **Dequeue** method, explain the meaning and purpose of the following code snippet!

```
if (front == max - 1) {
    front = 0;
}
```

4. In the **print** method, why does the loop variable **i** start from **front** instead of **0** (i.e., `int i = 0`)?
5. Review the **print** method again, and explain the meaning of the following code snippet!

```
i = (i + 1) % max;
```

6. Show the code snippet that represents a **queue overflow**!
7. When a queue overflow or underflow occurs, the program continues to run and only displays informational text. Modify the program so that it stops when a queue overflow or underflow happens!

Answer

1. front and rear are initialized to -1, size to 0: -1 means there is no valid element position yet (the queue is still empty), so it clearly distinguishes "no element" from real index 0. size counts the number of stored elements, so 0 naturally represents an empty queue. Both together mark the empty initial state.
2. `if (rear == max - 1) { rear = 0; }`: when rear has reached the last index of the array, it wraps back to index 0 instead of going out of bounds. This is the circular-queue behavior that reuses the empty slots freed at the front.
3. `if (front == max - 1) { front = 0; }`: the same circular logic for the front pointer. After a dequeue, if front is at the last index, it wraps back to 0 so traversal continues correctly around the array.
4. Loop starts from front, not 0: in a circular queue the active data does not always start at index 0. Starting from front ensures we read from the real first element; starting from 0 could print empty/old slots and miss the actual front.
5. `i = (i + 1) % max;`: moves the index forward by one position, and the modulo `% max` makes it wrap back to 0 after the last index — i.e., circular traversal of the array.
6. Queue overflow snippet is the else branch of enqueue, executed when `isFull()` is true:

```
}
    else if (isFull()) {
        System.out.println("Queue is full!");
    }
}
```

7. Stop on overflow/underflow

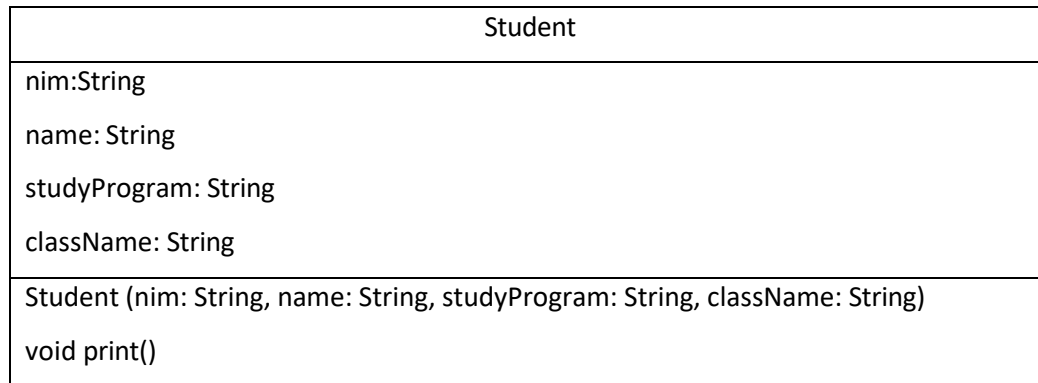
```
} else {  
    System.out.println("Queue is full!!! (overflow) - program stopped.");  
    System.exit(0);  
}
```

```
} else {  
    System.out.println("Queue is empty!!! (underflow) - program stopped.");  
    System.exit(0);  
}
```

2.2. Experiment 2 : Academic Service Queue

Allocated Time : 90 minutes

In this experiment, we will create a program that illustrates the service provided by the Academic Admin. Please refer to the following class diagram:



2.2.1. Experiment Steps

Based on the class diagram, a **Student** class will be created in Java.

1. Create a new class named **Student**
2. Add the attributes as shown in the class diagram. Then, create the parametric constructor as follows:

```
public Student(String nim, String name, String studyProgram, String
className){
    this.nim = nim;
    this.name = name;
    this.studyProgram = studyProgram;
    this.className = className;
}
```

Provide **print()** method to display the whole data of student.

```
void print(){
    System.out.println(nim+" - "+name+" - "+studyProgram+" -
"+className);
}
```

3. Copy the **Queue** class code from Experiment 1 to be reused in this experiment, and rename the class to **StudentQueue**. Since in Experiment 1, the data stored in the queue was only an **integer** array, while in this experiment the data used is **Student** object, you will need to modify the **StudentQueue** class accordingly.

```
Student[] data;
int front, rear, size, max;

public StudentQueue(int n){
    max = n;
    data = new Student[max];
    size = 0;
    front = 0;
    rear = -1;
}
```

4. Modify the **StudentQueue** class by changing the type of the data array from **int[]** to **Student[]**, since in this case the data to be stored consists of **Student** objects. The modifications should be applied to the attributes, the **enqueue** method, and the **dequeue** method.

```
void enqueue(Student dt){
    if(isFull()){
        System.out.println("Queue is full!!!");
        return;
    }
    rear = (rear+1)%max;
    data[rear] = dt;
    size++;
    System.out.printf("%s is successfully added at index %d\n",
dt.name, rear);
}

Student dequeue(){
    if(isEmpty()){
        System.out.println("Queue is empty!!!");
        return null;
    }
    Student dt = data[front];
    front = (front+1)%max;
    size--;
    return dt;
}
```

5. Next, implement the **peek** and **print** methods to display the front-most data in the service queue and to display all the data in the service queue, respectively.

```
void peek(){
```

```

        if(!isEmpty()){
            System.out.println("Front data: ");
            data[front].print();
        }else{
            System.out.println("Queue is empty!!!");
        }
    }

    void print(){
        if(isEmpty()){
            System.out.println("Queue is empty!!!");
            return;
        }
        int i = front;
        while(i != rear){
            data[i].print();
            i = (i+1)%max;
        }
        data[i].print();
        System.out.println("Number of element: "+size);
    }
}

```

6. Next, create a new class named **StudentQueueMain** within the same package. In this class, define the **main** function and declare a **Scanner** object with the name **sc**.
7. Then, instantiate an **StudentQueue** object named **queue** with the parameter set to the maximum queue size (for example, 5).
8. Declare an integer variable named **choice** to store the user's menu selection.
9. Add the following code to loop through the menu options based on the input provided by the user:

```

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    StudentQueue queue = new StudentQueue(5);
    int choice;
    do {
        System.out.println("\n=== Academic Service Menu ===");
        System.out.println("1. Enqueue Student");
        System.out.println("2. Dequeue Student");
        System.out.println("3. Display Front");
        System.out.println("4. Display All Data");
        System.out.println("5. Queue Size");
        System.out.println("0. Exit");
        System.out.print("Choose a menu: ");
        choice = sc.nextInt();
        sc.nextLine();
    }
}

```



```
switch (choice) {
    case 1:
        System.out.print("NIM   : ");
        String nim = sc.nextLine();
        System.out.print("Name : ");
        String name = sc.nextLine();
        System.out.print("Study Program : ");
        String studyProgram = sc.nextLine();
        System.out.print("Class : ");
        String className = sc.nextLine();
        Student std = new Student(nim, name, studyProgram,
className);

        queue.enqueue(std);
        break;
    case 2:
        Student studentBeingServed = queue.dequeue();
        if (studentBeingServed != null) {
            System.out.println("Student being served:");
            studentBeingServed.print();
        }
        break;
    case 3:
        queue.peek();
        break;
    case 4:
        queue.print();
        break;
    case 5:
        System.out.println("Queue size: " + queue.size);
        break;
    case 0:
        System.out.println("Thanks!!");
        break;
    default:
        System.out.println("Invalid menu!!");
}
} while (choice != 0);
}
```

10. Compile and run **StudentQueueMain** class and observe the result!



```

1  package Jobsheet11;
2
3  public class Student14 {
4      String nim, name, studyProgram, className;
5
6      public Student14(String nim, String name, String studyProgram, String className) {
7          this.nim = nim;
8          this.name = name;
9          this.studyProgram = studyProgram;
10         this.className = className;
11     }
12
13     void print() {
14         System.out.println(nim + " - " + name + " - " + studyProgram + " - " + className);
15     }
16 }
17

```

Jobsheet11 / StudentQueue14.java / StudentQueue14 / peek()

```

1  package Jobsheet11;
2
3  public class StudentQueue14 {
4      Student14[] data;
5      int front, rear, size, max;
6
7      public StudentQueue14(int n) {
8          max = n;
9          data = new Student14[max];
10         size = 0;
11         front = 0;
12         rear = -1;
13     }
14
15     boolean isEmpty() {
16         return size == 0;
17     }
18
19     boolean isFull() {
20         return size == max;
21     }
22
23     void enqueue(Student14 dt) {
24         if (isFull()) {
25             System.out.println(x: "Queue is full!!!");
26             return;
27         }
28         rear = (rear + 1) % max;
29         data[rear] = dt;
30         size++;
31         System.out.printf(format: "%s is successfully added at index %d\n", dt.name, rear);
32     }
33
34     Student14 dequeue() {
35         if (isEmpty()) {
36             System.out.println(x: "Queue is empty!!!");
37             return null;

```

```
33
34 ✓ Student14 dequeue() {
35 ✓     if (isEmpty()) {
36         System.out.println(x: "Queue is empty!!!");
37         return null;
38     }
39     Student14 dt = data[front];
40     front = (front + 1) % max;
41     size--;
42     return dt;
43 }
44
45 ✓ void peek() {
46 ✓     if (!isEmpty()) {
47         System.out.println(x: "Front data: ");
48         data[front].print();
49 ✓     } else {
50         System.out.println(x: "Queue is empty!!!");
51     }
52 }
53
54 ✓ void print() {
55 ✓     if (isEmpty()) {
56         System.out.println(x: "Queue is empty!!!");
57         return;
58     }
59     int i = front;
60 ✓     while (i != rear) {
61         data[i].print();
62         i = (i + 1) % max;
63     }
64     data[i].print();
65     System.out.println("Number of element: " + size);
66 }
```



```
1  package Jobsheet11;
2
3  import java.util.Scanner;
4
5  public class StudentQueueMain14 {
    Run | Debug
6      public static void main(String[] args) {
7          Scanner sc = new Scanner(System.in);
8          StudentQueue14 queue = new StudentQueue14(n: 5);
9          int choice;
10         do {
11             System.out.println(x: "\n=== Academic Service Menu ===");
12             System.out.println(x: "1. Enqueue Student");
13             System.out.println(x: "2. Dequeue Student");
14             System.out.println(x: "3. Display Front");
15             System.out.println(x: "4. Display All Data");
16             System.out.println(x: "5. Queue Size");
17             System.out.println(x: "0. Exit");
18             System.out.print(s: "Choose a menu: ");
19             choice = sc.nextInt();
20             sc.nextLine();
21             switch (choice) {
22                 case 1:
23                     System.out.print(s: "NIM : ");
24                     String nim = sc.nextLine();
25                     System.out.print(s: "Name : ");
26                     String name = sc.nextLine();
27                     System.out.print(s: "Study Program : ");
28                     String studyProgram = sc.nextLine();
29                     System.out.print(s: "Class : ");
30                     String className = sc.nextLine();
31                     Student14 std = new Student14(nim, name, studyProgram, className);
32                     queue.enqueue(std);
33                     break;
34                 case 2:
35                     Student14 studentBeingServed = queue.dequeue();
36                     if (studentBeingServed != null) {
```

```
33         break;
34     case 2:
35         Student14 studentBeingServed = queue.dequeue();
36         if (studentBeingServed != null) {
37             System.out.println(x: "Student being served:");
38             studentBeingServed.print();
39         }
40         break;
41     case 3:
42         queue.peek();
43         break;
44     case 4:
45         queue.print();
46         break;
47     case 5:
48         System.out.println("Queue size: " + queue.size);
49         break;
50     case 0:
51         System.out.println(x: "Thanks!!");
52         break;
53     default:
54         System.out.println(x: "Invalid menu!!");
55     }
56 } while (choice != 0);
57 sc.close();
58 }
59 }
```

2.2.2 Result Verification

Verify the output with the following:

```
=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
```



```
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 1
NIM    : 111
Name   : Joseph
Study Program : TI
Class  : 1I
Joseph is successfully added at index 0

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 1
NIM    : 222
Name   : Brian
Study Program : TI
Class  : 1I
Brian is successfully added at index 1

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 1
NIM    : 333
Name   : Herco
Study Program : TI
Class  : 1I
Herco is successfully added at index 2

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 1
NIM    : 444
Name   : Bima
Study Program : TI
Class  : 1I
Bima is successfully added at index 3

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 1
NIM    : 555
Name   : Kinan
Study Program : TI
Class  : 1I
Kinan is successfully added at index 4

=== Academic Service Menu ===
```



```
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 1
NIM    : 6666
Name   : Nahda
Study Program : TI
Class  : 1I
Queue is full!!!

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 5
Queue size: 5

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 4
111 - Joseph - TI - 1I
222 - Brian - TI - 1I
333 - Herco - TI - 1I
444 - Bima - TI - 1I
555 - Kinan - TI - 1I
Number of element: 5

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 3
Front data:
111 - Joseph - TI - 1I

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 2
Student being served:
111 - Joseph - TI - 1I

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
```



```

Choose a menu: 5
Queue size: 4

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu: 4
222 - Brian - TI - 1I
333 - Herco - TI - 1I
444 - Bima - TI - 1I
555 - Kinan - TI - 1I
Number of element: 4

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu:

```

```

PS C:\Users\pasap\OneDrive\Documents\Kuliah\Tugas\Semester 2\Praktik Algoritma Struktur Data\Jobsheet1-semester2> & 'C:\P
\Java\jdk-24\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\pasap\AppData\Roaming\Code\User\works
e0ae0740bf9eef90a1485cc4c4061a4f\redhat.java\jdt_ws\Jobsheet1-semester2_bc378dc3\bin' 'Jobsheet11.StudentQueueMain14'

=== Academic Service Menu ===
1. Enqueue Student
2. Dequeue Student
3. Display Front
4. Display All Data
5. Queue Size
0. Exit
Choose a menu:

```

2.2.3 Questions

1. What is the main difference between the **Queue** (experiment 1) and **StudentQueue** classes in terms of the **data type** they manage? Why is this change in data type important?
2. How do the **enqueue** and **dequeue** methods in **StudentQueue** differ from those in **Queue**?
3. Examine the **constructor** of the **StudentQueue** class: why is the **front** attribute initialized to 0, whereas in the **Queue** class from Experiment 1, the **front** attribute is initialized to -1?
4. Modify the program by adding a new method called **viewRear()** in the **StudentQueue** class to check the queue element at the **rear** position. Also, update the menu in the **StudentQueueMain** class by adding option 6: "Check rear of the queue," so that the **viewRear()** method can be invoked.

Answer

1. Main difference (data type): Queue manages a primitive `int[]`, while `StudentQueue` manages an object array `Student[]`, where each element carries several attributes (`nim`, `name`, `studyProgram`, `className`). This matters because a real queue usually holds complex records, not a single number, so one element can represent a full student entity.
2. enqueue/dequeue differences: `StudentQueue` uses a single circular expression — `rear = (rear+1)%max`; and `front = (front+1)%max`; — instead of separate if/else wrap checks. It also handles `Student` objects (returning null when empty) rather than `int` (returning 0), and prints `dt.name` instead of the integer value.
3. Why `front = 0` in `StudentQueue` (not -1): because positions are computed with the modulo formula and emptiness is tracked by `size`, every method guards with `isEmpty()` before reading `data[front]`, so starting `front` at 0 is safe and consistent with `rear = (rear+1)%max`. In Experiment 1, -1 was used as an explicit "no element yet" marker that fit its different (non-modulo) enqueue/dequeue logic.
4. Add `viewRear()` + menu option 6

```
44  
45  void viewRear() {  
46      if (!isEmpty()) {  
47          System.out.println(x: "Rear data: ");  
48          data[rear].print();  
49      } else {  
50          System.out.println(x: "Queue is empty!!!");  
51      }  
52  }  
53
```



2.3 Assignment

Time : 120 minutes

Create a queue-based program to simulate the approval queue for Student Course Registration Forms (KRS) by Academic Advisors (DPA). When a student joins the queue, they must first register their personal data (as in Experiment 2). Use queue classes similar to those used in Experiment1 and 2, and implement the following methods:

- Check if the queue is **empty**, check if the queue is **full**, and **clear** the queue.
- **Add** a student to the queue (enqueue), and process the KRS approval (dequeue) – each approval session processes **2 students at a time** (from the first two in the queue).
- Show **all students** in the queue, show the **first two** students in line, and show the **last student** in the queue.
- Print the **total number of students** in the queue, print the **number of students who have completed the KRS approval process**.
- The **maximum queue size is 10**, and each DPA can handle up to **30 students**. Display the number of students who have not yet completed the KRS approval process.

Draw the **class diagram** for the queue structure and implement all the required methods through a **menu-based interface** in the main function.



```

Jobsheet11 > Assignment > Student.java > Student > Student(String, String, String, String)
1  package Jobsheet11.Assignment;
2
3  public class Student {
4      String nim, name, studyProgram, className;
5
6      public Student(String nim, String name, String studyProgram, String className) {
7          this.nim = nim;
8          this.name = name;
9          this.studyProgram = studyProgram;
10         this.className = className;
11     }
12
13     void print() {
14         System.out.println(nim + " - " + name + " - " + studyProgram + " - " + className);
15     }
16 }
17
Jobsheet11 > Assignment > KRSQueue14.java > KRSQueue14 > print()
1  package Jobsheet11.Assignment;
2
3  public class KRSQueue14 {
4      Student[] data;
5      int front, rear, size, max;
6      int served; // jumlah mahasiswa yang KRS-nya sudah disetujui
7      final int dpaCapacity = 30; // setiap DPA menangani maksimal 30 mahasiswa
8
9      public KRSQueue14(int n) {
10         max = n;
11         data = new Student[max];
12         size = 0;
13         front = 0;
14         rear = -1;
15         served = 0;
16     }
17
18     boolean isEmpty() {
19         return size == 0;
20     }
21
22     boolean isFull() {
23         return size == max;
24     }
25
26     void clear() {
27         front = 0;
28         rear = -1;
29         size = 0;
30         System.out.println(x: "Queue has been cleared!");
31     }
32
33     void enqueue(Student dt) {
34         if (isFull()) {
35             System.out.println("Queue is full!!! (max " + max + " students)");
36             return;
37         }

```

```
33 void enqueue(Student dt) {
34     if (isFull()) {
35         System.out.println("Queue is full!!! (max " + max + " students)");
36         return;
37     }
38     rear = (rear + 1) % max;
39     data[rear] = dt;
40     size++;
41     System.out.println(dt.name + " joined the KRS approval queue.");
42 }
43
44 // Satu sesi persetujuan KRS = memproses 2 mahasiswa dari depan
45 void approveSession() {
46     if (isEmpty()) {
47         System.out.println(x: "Queue is empty!!! No student to approve.");
48         return;
49     }
50     System.out.println(x: "=== KRS Approval Session (up to 2 students) ===");
51     int count = Math.min(a: 2, size);
52     for (int k = 0; k < count; k++) {
53         Student dt = data[front];
54         front = (front + 1) % max;
55         size--;
56         served++;
57         System.out.print(s: "Approved: ");
58         dt.print();
59     }
60 }
61
62 void printAll() {
63     if (isEmpty()) {
64         System.out.println(x: "Queue is empty!!!");
65         return;
66     }
67 }
```

```

62     void printAll() {
63         if (isEmpty()) {
64             System.out.println(x: "Queue is empty!!!");
65             return;
66         }
67         System.out.println(x: "All students in queue:");
68         int i = front;
69         while (i != rear) {
70             data[i].print();
71             i = (i + 1) % max;
72         }
73         data[i].print();
74         System.out.println("Number of element: " + size);
75     }
76
77     void printFirstTwo() {
78         if (isEmpty()) {
79             System.out.println(x: "Queue is empty!!!");
80             return;
81         }
82         System.out.println(x: "First two students in line:");
83         int count = Math.min(a: 2, size);
84         int i = front;
85         for (int k = 0; k < count; k++) {
86             data[i].print();
87             i = (i + 1) % max;
88         }
89     }
90
91     void printLast() {
92         if (isEmpty()) {
93             System.out.println(x: "Queue is empty!!!");
94             return;
95         }
96         System.out.println(x: "Last student in queue:");
97         data[rear].print();
98     }
99
100    void info() {
101        System.out.println("Students currently in queue : " + size);
102        System.out.println("Students approved (completed): " + served);
103        System.out.println("Students not yet completed : " + (dpaCapacity - served));
104    }
105 }

```

```
1 package Jobsheet11.Assignment;
2
3 import java.util.Scanner;
4
5 public class KRSMain14 {
6     Run | Debug
7     public static void main(String[] args) {
8         Scanner sc = new Scanner(System.in);
9         KRSQueue14 queue = new KRSQueue14(n: 10); // maksimal ukuran queue = 10
10        int choice;
11        do {
12            System.out.println(x: "\n=== KRS Approval Queue (DPA) ===");
13            System.out.println(x: "1. Register & add student to queue");
14            System.out.println(x: "2. Process KRS approval (2 students)");
15            System.out.println(x: "3. Show all students in queue");
16            System.out.println(x: "4. Show first two in line");
17            System.out.println(x: "5. Show last student in queue");
18            System.out.println(x: "6. Show statistics (total/completed/not yet)");
19            System.out.println(x: "7. Check if queue is empty");
20            System.out.println(x: "8. Check if queue is full");
21            System.out.println(x: "9. Clear queue");
22            System.out.println(x: "0. Exit");
23            System.out.print(s: "Choose a menu: ");
24            choice = sc.nextInt();
25            sc.nextLine();
26            switch (choice) {
27                case 1:
28                    System.out.print(s: "NIM : ");
29                    String nim = sc.nextLine();
30                    System.out.print(s: "Name : ");
31                    String name = sc.nextLine();
32                    System.out.print(s: "Study Program : ");
33                    String studyProgram = sc.nextLine();
34                    System.out.print(s: "Class : ");
35                    String className = sc.nextLine();
36                    queue.enqueue(new Student(nim, name, studyProgram, className));
37                    break;
```

```
37  case 2:
38      queue.approveSession();
39      break;
40  case 3:
41      queue.printAll();
42      break;
43  case 4:
44      queue.printFirstTwo();
45      break;
46  case 5:
47      queue.printLast();
48      break;
49  case 6:
50      queue.info();
51      break;
52  case 7:
53      System.out.println(queue.isEmpty() ? "Queue is empty." : "Queue is NOT empty.");
54      break;
55  case 8:
56      System.out.println(queue.isFull() ? "Queue is full." : "Queue is NOT full.");
57      break;
58  case 9:
59      queue.clear();
60      break;
61  case 0:
62      System.out.println(x: "Thanks!!");
63      break;
64  default:
65      System.out.println(x: "Invalid menu!!");
66  }
67  } while (choice != 0);
68  sc.close();
69  }
70  }
```